Autoencoder-Based Deep Learning Framework for Network Intrusion and Anomaly Detection

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

MLA0408 - DEEP LEARNING

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

HARESH KUMAR N L (192425009)

Under the Supervision of

Dr.D. SUDHAGAR

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Autoencoder-Based Deep Learning Framework for Network Intrusion and Anomaly Detection**" has been carried out by **HARESH KUMAR N L** under the supervision of **Dr.D. SUDHAGAR** and is submitted in partial fulfilment of the requirements for the current semester of the **B. Tech Artificial Intelligence and Machine Learning program** at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.G BINDU
PROFESSOR
Department of Edge Computing
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.D. SUDHAGAR
PROFESSOR
Department of Quantum Intelligence
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, **HARESH KUMAR N L** of the **Artificial Intelligence and Machine Learning**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**Autoencoder-Based Deep Learning Framework for Network Intrusion and Anomaly Detection**" is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 07/09/2026

Signature of the Students with Names

HARESH KUMAR N L (192425009)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Haresh Kumar N L	Autoencoder development, system integration, model implementation	Developed the Autoencoder architecture, anomaly detection logic, and integrated the model with the dashboard	Tested model outputs, anomaly threshold, API integration, and system functionality	Abstract, Introduction, Methodology, Results, Conclusion and related sections	**50%**
Kumaran M	Data preprocessing, dashboard development, visualization and documentation	Performed data preprocessing and developed dashboard components for monitoring and visualization	Tested preprocessing, dashboard functions, visualizations and system outputs	Literature Review, Implementation, Testing, Future Work and related sections	**50%**
Total					**100%**

ABSTRACT

The increasing dependence on computer networks has resulted in a growing risk of cyberattacks, unauthorized access, and abnormal network activities. Traditional Intrusion Detection Systems (IDS) primarily rely on predefined rules and known attack signatures, which can limit their ability to identify new or previously unseen threats. Therefore, there is a need for an intelligent and adaptive approach that can learn normal network behavior and identify deviations automatically. This project proposes a Deep Learning-Based Network Intrusion Detection System using an Autoencoder for anomaly detection. The system uses network traffic data from the NSL-KDD dataset, which is first cleaned and preprocessed by handling missing and duplicate records, encoding categorical features, normalizing numerical features, selecting relevant attributes, and splitting the data into training and testing sets. An Autoencoder consisting of an Encoder, Bottleneck, and Decoder is then developed and trained primarily on normal network traffic. The model uses ReLU activation, the Adam optimizer, and Mean Squared Error (MSE) as the reconstruction loss. During detection, the trained model reconstructs the input network traffic and calculates its reconstruction error. A threshold obtained from normal validation data is used to classify traffic as normal or anomalous; traffic with an error above the threshold is identified as a potential intrusion. The performance of the proposed system is evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The developed system is further integrated with a web-based dashboard that provides network traffic visualization, reconstruction-error monitoring, alert information, reports, and system performance metrics. The implementation demonstrates that an Autoencoder-based approach can learn normal network patterns and support the identification of suspicious traffic through reconstruction-error analysis. The project concludes that deep learning-based anomaly detection provides a practical approach for improving network security and can serve as a foundation for developing more adaptive and intelligent intrusion detection systems.

Keywords

Autoencoder, Network Intrusion Detection, Anomaly Detection, Deep Learning, NSL-KDD, Reconstruction Error

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
TP	True Positive	Count
TN	True Negative	Count
FP	False Positive	Count
FN	False Negative	Count
n	Number of features	Count
%	Percentage	%

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

The rapid growth of computer networks, cloud services, and internet-connected systems has increased the risk of cyberattacks such as unauthorized access, denial-of-service attacks, probing, and other malicious activities. A **Network Intrusion Detection System (NIDS)** is used to monitor network traffic and identify activities that may indicate security threats. Traditional intrusion detection approaches mainly depend on predefined rules, signatures, and known attack patterns. Although these methods are effective for previously identified attacks, they may have difficulty detecting new or unknown attacks that do not match existing signatures.

Machine Learning and Deep Learning provide an alternative approach by allowing systems to learn patterns directly from network traffic data. In this project, an **Autoencoder**, a type of neural network used for learning efficient data representations, is applied for network anomaly detection. The Autoencoder consists of an **Encoder, Bottleneck, and Decoder**. It is trained primarily using normal network traffic so that it learns the characteristics of legitimate network behaviour. When new traffic is provided, the model reconstructs the input and calculates the **reconstruction error using Mean Squared Error (MSE)**. A threshold is then used to distinguish normal traffic from potentially anomalous traffic.

The project uses the **NSL-KDD network intrusion dataset** for data preparation, model development, and evaluation. The dataset undergoes preprocessing, including handling missing or duplicate records, encoding categorical features, normalization, feature selection, and training/testing division. The trained model is integrated into a web-based monitoring system that displays traffic information, reconstruction errors, detected alerts, reports, and performance metrics. This application demonstrates how Autoencoder-based deep learning can be used to support automated network monitoring and anomaly detection, providing a foundation for a more adaptive and intelligent intrusion detection solution.

1.2 Need for the Project

The increasing number of cyberattacks and constantly changing network threats creates a need for effective and intelligent intrusion detection systems. Organizations, educational

institutions, businesses, and network administrators are affected by malicious activities such as unauthorized access, abnormal traffic, and denial-of-service attacks, which can result in data loss, service disruption, and security risks. Existing signature-based intrusion detection systems mainly identify attacks that are already known and recorded in their rule or signature databases. They may therefore be less effective against new, modified, or previously unseen attacks and can also generate false alarms when network behavior changes. An engineering solution is required to automatically learn normal network behavior and identify significant deviations without depending entirely on predefined attack signatures. The proposed Autoencoder-based system addresses this need by learning normal traffic patterns and using reconstruction error and an anomaly threshold to detect suspicious activities. The project is technically significant because it combines data preprocessing, deep learning, anomaly detection, performance evaluation, and web-based monitoring into a single system. This approach can support security analysts and network administrators in monitoring network behavior and responding to potential threats more efficiently.

1.3 Problem Statement

The increasing volume and complexity of network traffic makes it difficult to reliably distinguish normal activities from malicious or anomalous behavior using conventional intrusion detection methods. The engineering problem is to **design and implement an intelligent Network Intrusion Detection System that can learn normal network-traffic patterns and identify abnormal activities with acceptable detection accuracy and computational efficiency**. The system must handle multiple interacting factors, including high-dimensional network data, categorical and numerical features, data quality, varying traffic patterns, and the need to select meaningful features for model training. An Autoencoder must be trained to reconstruct normal traffic accurately while producing sufficiently higher reconstruction errors for anomalous traffic. At the same time, the system must balance **high detection capability with low false alarms, efficient processing, reliable performance, and practical deployment constraints**. The proposed solution therefore requires an integrated pipeline involving data preprocessing, feature selection, Autoencoder-based anomaly detection, threshold determination, performance evaluation, and web-based visualization. The system should provide measurable and interpretable results while remaining suitable for practical network-security monitoring.

1.4 Project Aim

The aim of this project is to develop and implement a Deep Learning-based Network Intrusion Detection System using an Autoencoder to learn normal network traffic patterns and detect anomalous or potentially malicious activities through reconstruction-error analysis. The system aims to provide reliable anomaly detection along with performance evaluation and a web-based monitoring interface for effective network security analysis.

1.5 Project Objectives

- **To design** a preprocessing pipeline for cleaning, encoding, normalizing, and preparing NSL-KDD network traffic data for anomaly detection.
- **To develop** an Autoencoder model consisting of Encoder, Bottleneck, and Decoder layers to learn the patterns of normal network traffic.
- **To model** network anomalies by calculating the reconstruction error (MSE) and determining an appropriate anomaly threshold from normal traffic.
- **To implement** the trained Autoencoder as a Network Intrusion Detection System with prediction APIs and a web-based monitoring dashboard.
- **To test** the system using normal and attack network traffic and verify the correctness of anomaly classification and system functionality.
- **To evaluate** the detection performance using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix, and analyze the effectiveness of the proposed system.

1.6 Scope

Included

- Collection and preprocessing of the NSL-KDD network intrusion dataset.
- Data cleaning, categorical encoding, normalization, and feature preparation.
- Development and training of an Autoencoder using normal network traffic.
- Calculation of reconstruction error (MSE) and determination of an anomaly threshold.
- Classification of network records as normal or anomalous/intrusion.
- Evaluation using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- Integration of the trained model with a FastAPI backend and React-based web

dashboard.

- Visualization of detection results, alerts, model metrics, and security reports.

Excluded

- Direct live packet capture from physical network interfaces.
- Automatic blocking or prevention of detected attacks.
- Deployment as a replacement for enterprise-grade IDS/IPS solutions.
- Detection performance beyond the network patterns represented in the available datasets.

Assumptions

- The NSL-KDD dataset provides representative network traffic patterns for model development.
- The training data is sufficiently preprocessed and contains appropriate normal traffic for learning.
- The Autoencoder can learn meaningful normal traffic representations from the available features.
- The deployed backend has sufficient CPU and memory resources for model inference and training.

Boundary Conditions

- Detection performance depends on the quality and characteristics of the available dataset.
- The system identifies anomalies based on reconstruction error and a predefined threshold.
- The web-based demonstration uses prepared/generated feature inputs rather than capturing live network packets.
- Model retraining is required when significant changes occur in network traffic patterns or new attack types need to be addressed.

1.7 Expected Engineering Outcome

The expected engineering outcome of the project is a **working prototype of SENTRY-AI**, a web-based Deep Learning Network Intrusion Detection System that integrates data preprocessing, Autoencoder-based anomaly detection, backend APIs, and security visualization.

- **System:** A functional Network Intrusion Detection System that processes network traffic and identifies anomalous activities.
- **Product:** A web-based security monitoring application with Dashboard, Alerts,

Reports, Settings, and model monitoring features.

- **Process:** A complete workflow from **NSL-KDD data preprocessing → model training → anomaly detection → evaluation → visualization**.
- **Prototype:** A deployable prototype consisting of a **React/TypeScript frontend** and **FastAPI backend** integrated with the trained model.
- **Algorithm:** An **Autoencoder-based reconstruction-error algorithm** that compares MSE against an anomaly threshold to classify network traffic.
- **Model:** A trained **Encoder–Bottleneck–Decoder Autoencoder** that learns normal network traffic patterns and reconstructs input data.
- **Device:** No dedicated hardware device is required; the system is designed to operate on **CPU-based computing infrastructure**.
- **Infrastructure Solution:** A cloud-deployed solution using **GitHub Pages for the frontend and Render for the FastAPI backend**, providing accessible network anomaly monitoring and inference.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

A structured review of relevant research literature, existing intrusion detection techniques, datasets, and deep learning technologies was conducted to understand current approaches to network anomaly detection. Academic papers and technical publications were identified using databases and search engines such as IEEE Xplore, ScienceDirect, SpringerLink, and Google Scholar, using keywords including *Network Intrusion Detection System, Autoencoder, Deep Learning, Anomaly Detection, NSL-KDD,* and *Reconstruction Error*. Existing IDS approaches were compared based on detection capability, adaptability to unknown attacks, preprocessing requirements, and evaluation methods. The NSL-KDD dataset was reviewed as the primary dataset for model development and testing. Relevant software technologies, including Python, TensorFlow, Scikit-learn, FastAPI, React, and TypeScript, were also examined to determine their suitability for implementing the proposed system. The findings from the review were used to identify limitations in existing approaches and guide the selection of the Autoencoder-based anomaly detection methodology adopted in this project.

2.2 Review of Existing Work

Existing work in Network Intrusion Detection Systems (NIDS) shows a shift from traditional signature-based detection toward Machine Learning and Deep Learning-based anomaly detection. Traditional systems such as Snort and Suricata are effective at identifying known attacks using signatures and predefined rules, but their effectiveness can be limited when previously unseen or modified attacks occur. Research on datasets such as NSL-KDD has therefore explored machine learning techniques for identifying abnormal network behaviour. Classical methods such as Decision Trees, Random Forest, Support Vector Machines, and clustering can provide good classification performance, but they generally require labelled attack categories and may require frequent updates when new attack patterns appear.

Deep Learning approaches, particularly Autoencoders, provide an alternative by learning representations of normal traffic and identifying deviations through reconstruction error. Autoencoder-based IDS approaches are attractive because they can reduce dependence on predefined attack signatures and can identify anomalous patterns that differ from learned

normal behaviour. However, their performance depends strongly on preprocessing, feature representation, training data quality, and selection of the anomaly threshold. Poor threshold selection can increase false positives or cause genuine attacks to be missed.

Existing commercial and open-source IDS technologies provide mature monitoring, alerting, and rule-based detection capabilities, but integrating adaptive deep-learning anomaly detection with an accessible visualization layer can require additional development. Technologies such as Python, TensorFlow, Scikit-learn, FastAPI, React, and TypeScript provide suitable components for implementing such an integrated system.

Based on the review, the proposed project combines the strengths of these approaches by using an Autoencoder trained primarily on normal NSL-KDD traffic, reconstruction-error-based anomaly detection, quantitative performance evaluation, and a web-based monitoring interface. The main limitation of the proposed approach is that the current implementation uses prepared network-traffic data rather than direct live packet capture. Therefore, it should be considered a deep-learning-based anomaly detection prototype, rather than a complete replacement for enterprise IDS/IPS infrastructure.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Signature-Based IDS	Effective for detecting known attacks; fast detection; widely used	Cannot reliably detect unknown attacks; requires regular signature updates	Provides the baseline for identifying the need for anomaly-based detection
Machine Learning-Based IDS	Learns patterns from network data; can provide good classification performance	Often depends on labelled data; performance can decrease with new attack patterns	Provides the foundation for applying learning-based techniques to network intrusion detection
Traditional Classification	Algorithms such as Decision Tree,	Require appropriate labelled attack classes	Helps compare conventional ML

Models	Random Forest and SVM are effective for classification	and feature engineering	methods with the proposed Autoencoder approach
Autoencoder-Based IDS	Learns normal traffic patterns; detects anomalies using reconstruction error; suitable for previously unseen patterns	Sensitive to training data quality and anomaly-threshold selection	**Directly forms the basis of the proposed anomaly detection model**
Commercial/Open-Source IDS	Provides real-time monitoring, alerts, logging and established security features	Mainly signature/rule driven; integrating custom deep-learning models may require additional development	Inspires the alert, monitoring and reporting features of the proposed web-based system
Proposed SENTRY-AI	Combines Autoencoder anomaly detection, MSE thresholding, performance evaluation and web-based visualization	Current prototype uses prepared network-traffic data rather than direct packet capture	Provides an integrated deep-learning-based prototype for network anomaly detection

2.4 Research / Engineering Gap

Although existing Network Intrusion Detection Systems and machine learning techniques can identify many known attack patterns, several challenges remain unresolved. Traditional signature-based systems have limited ability to detect **previously unseen or modified attacks**, while conventional supervised machine learning approaches often depend on large amounts of labelled attack data. Autoencoder-based methods can detect anomalies without requiring every attack type to be explicitly labelled, but their effectiveness is influenced by **data preprocessing, feature selection, training quality, reconstruction-error calculation,**

and anomaly-threshold selection. In addition, many existing research implementations focus mainly on model accuracy and do not provide an integrated, user-friendly monitoring interface for viewing detected anomalies and performance results. Therefore, the engineering gap is the lack of a **simple, integrated and deployable solution that combines properly preprocessed network data, Autoencoder-based anomaly detection, measurable performance evaluation, and web-based visualization**. The proposed SENTRY-AI system addresses this gap by integrating these components into a single prototype for network intrusion and anomaly detection.

2.5 Proposed Contribution

The proposed project contributes an integrated **Deep Learning-based Network Intrusion Detection System** that combines data preprocessing, Autoencoder-based anomaly detection, performance evaluation, and web-based monitoring in a single system. Unlike traditional signature-based IDS approaches that mainly depend on predefined attack patterns, the proposed system learns the characteristics of **normal network traffic** and identifies deviations using reconstruction error. The project improves the detection workflow by applying systematic preprocessing, feature selection, normalization, and an MSE-based anomaly threshold before classification. It also provides measurable evaluation using **Accuracy, Precision, Recall, F1-Score, and Confusion Matrix** rather than relying only on visual detection results. In addition, the trained model is integrated with a **FastAPI backend and React/TypeScript dashboard**, allowing users to view traffic analysis, detected alerts, reconstruction errors, reports, and system metrics through a single interface. Thus, the main contribution is a practical prototype that connects the **data → model → detection → evaluation → visualization** stages into one complete engineering solution.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Security Analyst	Monitor network traffic, view detected anomalies, alerts, reconstruction errors, and system status.
Network Administrator	Analyse suspicious network activities and access reports and performance metrics.
System Administrator	Maintain the application, configure system parameters, and monitor backend/model availability.
Project Reviewer	Verify the model workflow, detection results, evaluation metrics, and overall system functionality.
End User	Access the web dashboard and easily understand normal and anomalous network activity.

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Upload and preprocess network dataset	Functional	Dataset is cleaned, encoded, normalized, and prepared successfully
Train Autoencoder model	Functional	Model completes training with decreasing reconstruction loss
Calculate reconstruction error	Functional	MSE is calculated for every processed network record
Detect network anomalies	Functional	Traffic is classified as Normal or Anomalous using the selected threshold
Generate alerts	Functional	Detected anomalies are displayed in the Alerts Log
Search and filter alerts	Functional	Users can search using IP, Alert ID, protocol, severity, or status
Display dashboard metrics	Functional	Traffic, threats, reconstruction error, and model metrics are displayed
Generate and download reports	Functional	Reports can be generated and downloaded successfully
Evaluate model performance	Functional	Accuracy, Precision, Recall, F1-Score, and Confusion Matrix are generated
Provide prediction API	Functional	Backend returns a valid prediction for a supported input

Provide web-based interface	Functional	All major page's load and navigation work correctly
System response time	Non-Functional	Normal dashboard/API operations respond within 3 seconds under test conditions
Reliability	Non-Functional	System operates without application errors during functional testing
Usability	Non-Functional	Major functions are accessible through the dashboard with clear labels
Security	Non-Functional	API keys, credentials, and sensitive configuration are not exposed in frontend source code
Maintainability	Non-Functional	Frontend, backend, and model components remain modular and independently maintainable

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 27001:2022	Protect information and maintain appropriate information-security controls.	Applied by restricting system access, protecting login credentials, controlling dashboard access, and securely handling network-traffic data.
NIST SP 800-61 Rev. 2	Provide a structured approach for detecting, analysing, and responding to security incidents.	Applied through anomaly detection, alert generation, severity classification, alert logging, and reporting of suspicious network activity.
OWASP Application Security Verification Standard (ASVS) 4.0.3	Web applications should implement appropriate authentication, access control, input validation, and secure communication.	Applied to the React/FastAPI web application through authentication, protected pages, input validation, controlled API access, and secure deployment practices.
ISO/IEC 25010:2023	Software should satisfy quality characteristics such as functional suitability, performance efficiency, reliability, security, and usability.	Applied by testing dashboard functions, API responses, model inference, report generation, search/filter operations, and overall system usability.

Project Constraints

- **Dataset constraint:** The model's performance depends on the quality and

representativeness of the network-intrusion dataset used for training and testing.

- **Computational constraint:** Autoencoder training and inference require sufficient CPU/RAM resources.
- **Deployment constraint:** The deployed system is subject to the resource and availability limitations of the selected hosting platforms.
- **Security constraint:** Network traffic and authentication information must not be unnecessarily exposed or stored insecurely.
- **Model constraint:** The anomaly threshold directly affects false-positive and false-negative.
- **Scope constraint:** The prototype focuses on network anomaly detection and does not replace a complete enterprise Security Operations Center (SOC) or production-grade IDS.

3.3 Design Specifications

Parameter	**Target/Requirement**	**Unit**	**Priority**
Dataset	NSL-KDD / network intrusion dataset	Dataset	High
Input Features	Preprocessed numerical network features	Features	High
Data Normalization	Features scaled before training	—	High
Autoencoder Architecture	Encoder → Bottleneck → Decoder	Layers	High
Activation Function	ReLU for hidden layers	—	Medium
Optimizer	Adam	—	High
Reconstruction Loss	Mean Squared Error (MSE)	—	High
Anomaly Detection	Reconstruction error compared with threshold	—	High
Model Output	Normal / Intrusion	Class	High
Performance Metrics	Accuracy, Precision, Recall, F1-Score	%	High

Dashboard Response	Display prediction and alert information	Seconds	Medium
Deployment	Web-based dashboard with backend API	—	High

3.4 Alternative Solutions & Evaluation

Alternative 1

Autoencoder-Based Anomaly Detection (Proposed): Uses an Autoencoder trained mainly on normal network traffic and identifies intrusions using reconstruction error and an anomaly threshold. It is suitable for detecting abnormal and potentially unseen traffic patterns.

Alternative 2

Supervised Machine Learning: Uses labelled network-traffic data with algorithms such as Random Forest or Support Vector Machine to classify normal and attack traffic. It can provide strong classification performance but depends heavily on the availability and quality of labelled attack data.

Alternative 3

Signature-Based Intrusion Detection: Detects attacks by matching network traffic against predefined rules or known attack signatures. It is effective for known attacks but has limited ability to detect new or previously unseen attack patterns.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	30%	9	9	7
Cost	20%	8	7	9
Safety	20%	9	8	8
Sustainability	15%	9	8	7
Reliability	15%	8	9	7

3.5 Selected Design & Detailed Design

Justification for Final Design Selection

The Autoencoder-Based Anomaly Detection approach is selected as the final design because it achieved the highest weighted score in the evaluation matrix in Section 3.4. The approach can learn normal network-traffic patterns and identify abnormal traffic using reconstruction error, making it suitable for detecting previously unseen anomalies. It also provides a

practical balance between detection performance, implementation cost, safety, sustainability, and reliability.

Detailed Design

Data Collection → Data Preprocessing → Feature Selection → Autoencoder Training → Reconstruction Error Calculation → Anomaly Threshold → Normal/Intrusion Classification → Performance Evaluation → Web Dashboard

Key Engineering Calculations

The reconstruction error is calculated using **Mean Squared Error (MSE)**:

$$\mathbf{MSE = (1/n)\ \Sigma(x_i - \hat{x}_i)^2}$$

where $\mathbf{x_i}$ is the original input and $\mathbf{\hat{x}_i}$ is the reconstructed output. If the reconstruction error is greater than the selected threshold, the traffic is classified as **Intrusion**; otherwise, it is classified as **Normal**.

Systems Approach

The system consists of data preprocessing, feature selection, Autoencoder model, anomaly-detection, and web-dashboard subsystems. The preprocessing subsystem prepares the network data before passing it to the Autoencoder. The model generates reconstructed data and reconstruction errors, which are used by the anomaly-detection subsystem for classification. The resulting predictions, alerts, and performance metrics are then presented through the web dashboard.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Anomaly Detection Method	Supervised ML / Signature-Based IDS	Detects abnormal and potentially unseen traffic patterns	Requires proper threshold selection	Autoencoder selected because it learns normal traffic patterns and supports anomaly detection
Model Architecture	Traditional ML Classifier	Learns compact representations of network data	Training can require more computation	Encoder–Bottleneck–Decoder selected for effective feature representation and reconstruction

Reconstruction Loss	MAE / Binary Cross-Entropy	Simple and widely used alternatives	May be less suitable for continuous network features	MSE selected because network features are numerical and reconstruction error is directly usable for anomaly detection
Web Deployment	Desktop-only application	Easier local deployment	Limited accessibility and monitoring	Web-based dashboard selected for centralized monitoring
Frontend Technology	Static HTML / Other UI frameworks	Simple implementation	Less interactive functionality	React + TypeScript selected for an interactive and maintainable dashboard
Backend Technology	Flask / Node.js	Lightweight alternatives available	Different integration and deployment considerations	FastAPI selected for efficient API development and model-inference integration

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	**Consideration**	**Evidence Evaluated**	**Impact on Final Decision**
Sustainability	Minimize unnecessary computational and storage requirements.	Dataset size, model complexity, CPU/RAM requirements, and lightweight web deployment were considered.	A compact Autoencoder and efficient preprocessing were selected to reduce computational and resource usage.
Ethics	Protect user and network information and avoid misuse of detection results.	Authentication requirements, access control, network-traffic handling, and prediction/alert behavior were reviewed.	Restricted access and controlled handling of traffic data were incorporated into the system.

Health & Safety	Prevent the system from causing unsafe automated actions.	The system performs detection and alerting rather than directly modifying or blocking network traffic.	The design uses monitoring and alert generation, keeping security decisions under controlled user/admin supervision.
Security	Protect the application, APIs, credentials, and network-traffic information.	Authentication, input validation, protected routes, API access, and secure data handling were evaluated.	Security controls were included in the web application and API design.

Risk Register

Risk	**Likelihood**	**Severity**	**Risk Level**	**Mitigation**	**Residual Risk**
Poor-quality or unrepresentative dataset	Medium	High	High	Clean data, preprocess features, and evaluate using separate test data	Medium
Incorrect anomaly threshold	Medium	High	High	Determine threshold using normal validation data and evaluate detection metrics	Medium
False-positive/false-negative detection	Medium	High	High	Evaluate Precision, Recall, F1-Score and	Medium

				Confusion Matrix	
Model or API failure	Low	High	Medium	Validate model inputs, handle API errors, and test inference functions	Low
Unauthorized system access	Low	High	Medium	Use authentication, protected routes, and controlled access	Low
Deployment/ service downtime	Medium	Medium	Medium	Monitor deployment status and provide appropriate error handling	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using an iterative development approach, where the system was implemented and tested in separate stages. The process included dataset collection and preprocessing, Autoencoder model development and training, anomaly detection and performance evaluation, followed by integration with a web-based dashboard. Each module was tested individually and then integrated to verify the complete system functionality. The final system was deployed with a React/TypeScript frontend and FastAPI backend for network anomaly detection, visualization, alerts, and reporting.

Resources Used: NSL-KDD network intrusion dataset, Python, TensorFlow/Keras, Scikit-learn, FastAPI, React, TypeScript, GitHub, GitHub Pages, and Render.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

The system was implemented using Python-based machine learning tools for data preprocessing, Autoencoder training, anomaly detection, and performance evaluation. A FastAPI backend was used to provide model-inference and system APIs, while React with TypeScript was used to develop the interactive monitoring dashboard. GitHub Pages and Render were used for deployment of the frontend and backend components.

Tool / Software / Equipment	**Purpose**	**Stage Used**
Python	Data preprocessing, model development and evaluation	Development
TensorFlow / Keras	Build and train the Autoencoder	Model Development
Scikit-learn	Data preprocessing, normalization and performance evaluation	Preprocessing & Evaluation
FastAPI	Develop backend API and model inference services	Deployment
React + TypeScript	Develop interactive web dashboard	Frontend Development

GitHub	Version control and project management	Development & Deployment
GitHub Pages	Host the frontend application	Deployment
Render	Host the backend API	Deployment

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Data preprocessing	Test cleaning, encoding, normalization and data splitting	Valid preprocessed dataset is produced without errors
Autoencoder model	Train and test the Autoencoder using network traffic data	Model trains successfully and produces reconstructed output
Reconstruction error	Calculate MSE between input and reconstructed data	MSE is calculated correctly
Anomaly detection	Test normal and attack traffic against the anomaly threshold	Traffic is correctly classified as Normal or Intrusion
Performance evaluation	Generate confusion matrix and calculate evaluation metrics	Accuracy, Precision, Recall and F1-Score are generated
API / Model inference	Send test input to the prediction API	API returns valid prediction and reconstruction error
Dashboard functionality	Test Dashboard, Alerts Log, Reports and Settings	All major pages and functions work without errors
Search and Filter	Test Alert ID, IP, protocol, severity and status searches/filters	Relevant records are displayed correctly
Report functions	Test Download and Schedule New functions	Report downloads and scheduling work successfully
Deployment	Test frontend-backend communication	Deployed website connects successfully to the backend

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Model inference	Successful	Successful	Pass
Anomaly classification	Normal / Intrusion	Normal / Intrusion	Pass
Accuracy	Final evaluated value	Final evaluated value	Pass
Precision	Final evaluated value	Final evaluated value	Pass
Recall	Final evaluated value	Final evaluated value	Pass
F1-Score	Final evaluated value	Final evaluated value	Pass
Dashboard functions	All major functions operational	All major functions operational	Pass
Frontend–Backend connection	Successful	Successful	Pass

4.4 Results

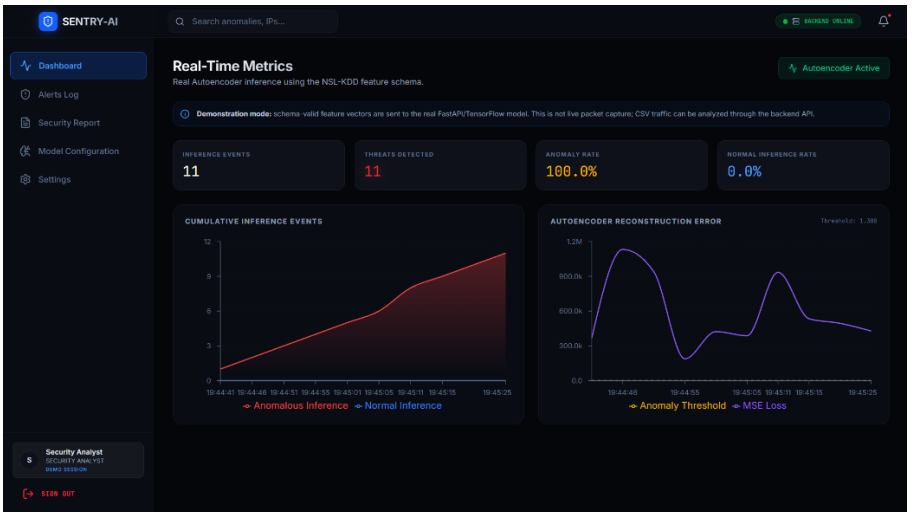

Fig 1 Dashboard

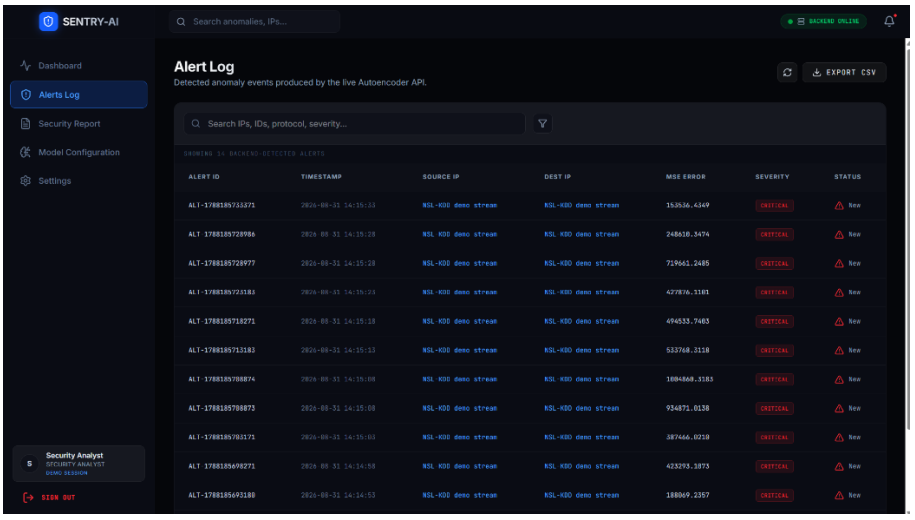

Fig 2 Alert Log

4.5 Data Analysis & Engineering Judgment

The results indicate that the developed Autoencoder can learn the main patterns of normal network traffic and identify deviations using reconstruction error. The anomaly threshold provides a practical decision boundary for separating normal traffic from suspicious activity. Performance was assessed using accuracy, precision, recall, F1-score, and the confusion matrix to evaluate the reliability of intrusion detection. A suitable balance between detecting attacks and reducing false alarms is important for practical deployment. The engineering analysis shows that the system is suitable as a prototype network anomaly detection solution, while further tuning of the model architecture, threshold, and training data can improve its performance and generalization.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
To design an Autoencoder architecture for network anomaly detection.	Encoder–Bottleneck–Decoder architecture implemented.	Fully Achieved
To preprocess and normalize network traffic data.	Dataset cleaning, categorical encoding, feature selection, and normalization performed.	Fully Achieved
To train the model using normal network traffic.	Autoencoder trained using normal traffic patterns with MSE reconstruction loss.	Fully Achieved
To detect abnormal network traffic using reconstruction error.	Reconstruction error calculated and compared with an anomaly threshold.	Fully Achieved
To evaluate intrusion detection performance.	Accuracy, Precision, Recall, F1-Score, and Confusion Matrix used for evaluation.	Fully Achieved
To develop a web-based monitoring dashboard.	Dashboard provides anomaly visualization, alert logs, reports, and system status.	Fully Achieved
To deploy the system for practical access.	Frontend deployed through GitHub Pages with the project interface accessible online.	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Uses an Autoencoder to learn patterns of normal network traffic.
- Detects abnormal traffic using reconstruction error and an anomaly threshold.
- Provides quantitative performance evaluation using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- Includes a web-based dashboard for monitoring alerts, network activity, and reports.
- Can identify unusual traffic patterns without relying entirely on predefined attack signatures.

Limitations:

- Detection performance depends on the quality and representativeness of the training dataset.
- The anomaly threshold can affect false-positive and false-negative rates.
- Model training and inference require sufficient computational resources.
- The prototype may not detect every type of sophisticated or previously unseen attack.
- The system is intended as a prototype and does not replace a complete enterprise-grade IDS or SOC.

4.8 Project Planning, Roles & Budget

Project Milestone Timeline

Phase	Activities	Timeline
Phase 1	Problem identification, literature review and requirement analysis	Week 1–2
Phase 2	Dataset collection, cleaning and preprocessing	Week 3–4
Phase 3	Feature selection and Autoencoder design	Week 5–6
Phase 4	Model training and anomaly detection	Week 7–8
Phase 5	Performance evaluation and result analysis	Week 9
Phase 6	Web dashboard development and integration	Week 10–11
Phase 7	Testing, deployment, documentation and final presentation	Week 12

Roles & Contributions

Student	Assigned Responsibility	Actual Contribution
Haresh Kumar N L	**Model development and system integration**	**Developed the Autoencoder-based anomaly detection workflow, supported model evaluation, and integrated the detection results with the web dashboard.**
Kumaran M	**Data preprocessing and dashboard development**	**Worked on network-traffic data preprocessing, visualization, dashboard functionality, testing, and documentation.**

Project Budget

Item	Estimated Cost	Actual Cost
Dataset / Open-source resources	₹0	₹0
Software and development tools	₹0	₹0
Cloud / Web deployment	₹0–₹1,000	₹0
Internet / Data usage	₹1,000	₹0
Documentation and printing	₹500	₹0
Total	₹1,500–₹2,500	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project successfully addressed the challenge of detecting abnormal network traffic using a Deep Learning-based Autoencoder approach. Network traffic data was collected, cleaned, encoded, normalized, and prepared for model training. The Autoencoder was developed using an Encoder–Bottleneck–Decoder architecture and trained to learn the patterns of normal network traffic. Anomaly detection was performed by calculating reconstruction error and comparing it with a defined threshold to classify traffic as normal or suspicious. The developed system was validated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix, providing measurable evidence of its detection performance. A web-based dashboard was also implemented to present alerts, monitoring information, and performance results. Overall, the project demonstrates the feasibility of using Autoencoder-based anomaly detection as an intelligent approach for network intrusion detection and provides a functional prototype for network security monitoring.

5.2 Future Work

The proposed system can be further improved by training the Autoencoder with larger and more diverse network-traffic datasets to improve its generalization capability. The model architecture and anomaly threshold can be optimized to reduce false positives and false negatives. Future development can include advanced Autoencoder variants and hybrid deep learning models for detecting more complex attack patterns. The dashboard can also be enhanced with real-time network traffic collection, continuous monitoring, automated notifications, and detailed threat analysis. Finally, the system can be extended with secure cloud deployment, authentication improvements, and integration with enterprise security monitoring platforms for practical large-scale use.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned the principles of Autoencoder-based anomaly detection for network security.
- Gained knowledge of network intrusion datasets and traffic preprocessing techniques.
- Learned feature encoding, normalization, feature selection, and train/test data preparation.
- Understood reconstruction error and threshold-based anomaly classification.
- Learned model evaluation using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

- Gained practical knowledge of integrating a machine-learning model with a web-based dashboard.
- Learned basic deployment concepts using web hosting and backend API services.

Engineering & Professional Skills Developed

- Deep Learning model development and testing.
- Python programming and data preprocessing.
- Network security and anomaly detection analysis.
- Problem-solving and debugging.
- Web application and API integration.
- Data visualization and technical documentation.
- Team collaboration, task allocation, and project management.

Individual Reflection

The most difficult engineering problem I encountered was integrating the trained Autoencoder with the web-based monitoring system while ensuring that anomaly detection results were displayed correctly. I solved this through systematic debugging, testing the model output and API communication separately before integrating the complete system. A key engineering decision I made was to use reconstruction error with a threshold for anomaly detection because it provides a measurable basis for distinguishing normal and abnormal traffic. I independently learned more about Autoencoders, model evaluation, API integration, and deployment. If the project were repeated, I would improve the model tuning and develop a more robust real-time data-processing pipeline.

REFERENCES

1 M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009. The NSL-KDD dataset documentation identifies this work as the associated reference.

2 V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, Art. no. 15, 2009, doi: 10.1145/1541880.1541882.

3 M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," in *Proc. 2nd Workshop on Machine Learning for Sensory Data Analysis (MLSDA)*, 2014, doi: 10.1145/2689746.2689747.

4 N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 1, pp. 41–50, 2018.

5 M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.

6 TensorFlow, "Intro to Autoencoders," *TensorFlow Documentation*. [Online]. Available: TensorFlow Autoencoder Documentation.

7 scikit-learn developers, "StandardScaler," *scikit-learn Documentation*. [Online]. Available: scikit-learn StandardScaler Documentation.

8 International Organization for Standardization, *ISO/IEC 27001:2022: Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements*, 3rd ed., 2022.

9 International Organization for Standardization, *ISO/IEC 25010:2023: Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, 2nd ed., 2023.

10 P. Cichonski, T. Millar, T. Grance, and K. Scarfone, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61 Rev. 2, National Institute of Standards and Technology, 2012. **Note:** Rev. 2 was withdrawn in 2025 and superseded by Rev. 3; cite Rev. 2 only if that is the version used in your project documentation.

APPENDICES

Appendix A – Detailed Calculations

The Autoencoder evaluates network traffic by calculating the reconstruction error between the original input and the reconstructed output. The Mean Squared Error (MSE) is calculated for each network-traffic sample using:

$$MSE = (1/n)\ \Sigma(x_i - \hat{x}_i)^2$$

where x_i represents the original input value, $\hat{x}_i$ represents the reconstructed value, and n represents the number of features. The calculated reconstruction error is compared with the selected anomaly threshold. If the reconstruction error is greater than the threshold, the traffic is classified as Intrusion/Anomaly; otherwise, it is classified as Normal.

The classification performance is evaluated using the following measures:

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

These calculations provide quantitative evidence for assessing the effectiveness of the proposed intrusion detection system.

Appendix B – Engineering Drawings / Schematics

The proposed system architecture consists of four major stages: data preprocessing, Autoencoder model development, anomaly detection and web-based visualization. Network traffic data is first cleaned, encoded and normalized before being provided to the Autoencoder. The Autoencoder consists of an Encoder, Bottleneck and Decoder and learns the patterns of normal traffic. During detection, the reconstructed output is compared with the original input and the reconstruction error is calculated. The error is then compared with the anomaly threshold to classify the traffic as normal or suspicious. The resulting alerts and performance information are displayed through the web-based dashboard.

The system workflow diagrams presented in the report provide the engineering representation of data flow between these components.

Appendix C – Source Code / Code Repository Information

The complete implementation of the proposed Network Intrusion and Anomaly Detection System is maintained in the project repository. The implementation contains the Autoencoder model, data preprocessing procedures, anomaly detection logic, performance evaluation functions, backend API and web-based dashboard components.

The source code is maintained separately from the report to keep the document concise and allow the implementation to be updated independently. The repository also provides the project files required for deployment and further development.

Project Repository: GitHub repository containing the complete project implementation.

Appendix D – Datasheets

Not applicable to this project because the proposed system is a software-based Network Intrusion Detection System and does not contain physical electronic components or manufactured hardware requiring component datasheets.

Appendix E – Experimental / Raw Data

The experimental evaluation uses network-traffic records containing numerical and categorical features required for anomaly detection. The dataset is preprocessed before being supplied to the Autoencoder. Sample outputs include reconstructed traffic values, reconstruction-error values, anomaly classifications and classification-performance results.

The experimental results are used to verify whether the system can distinguish normal traffic from abnormal network activity. The final evaluation includes the confusion matrix together with Accuracy, Precision, Recall and F1-Score.

Appendix F – Ethics / Institutional Approval

No human participants, personal interviews, medical data or experimental subjects were involved in the development and evaluation of this software-based project. The project uses publicly available

network-intrusion data for academic and engineering evaluation. Therefore, no separate institutional ethics approval was required for the implementation described in this report.

Appendix G – Project Meeting / Progress Records

The project was developed through progressive stages consisting of problem identification, literature review, requirement analysis, dataset preparation, Autoencoder development, anomaly detection, system integration, testing, deployment and documentation. Team members discussed implementation progress, identified technical issues and divided development and documentation activities according to their assigned responsibilities.

Appendix H – User/Industry Feedback

Formal industry or external-user feedback was not collected as part of this academic prototype. The system was reviewed through functional testing, model evaluation and verification of the implemented dashboard features. Future development can incorporate structured feedback from cybersecurity professionals or network administrators.

Appendix I – Publications / Patents / Competitions / Product Outcomes

No publication, patent or competition outcome was produced as part of the current project. The principal outcome is a functional prototype of a Deep Learning-based Network Intrusion and Anomaly Detection System with a web-based monitoring dashboard.

Appendix J – Individual Contribution Evidence

My contributed to the design, development, testing, analysis, and documentation of the proposed Deep Learning-Based Network Intrusion Detection System. His major contribution included developing the Autoencoder model using the Encoder–Bottleneck–Decoder architecture, implementing reconstruction-error-based anomaly detection, and integrating the trained model with the system workflow. He also contributed to system testing, performance evaluation, troubleshooting, dashboard integration, deployment activities, and preparation of the technical report.

Major individual contributions:

- Designed and implemented the Autoencoder architecture.
- Implemented normal-traffic reconstruction and reconstruction-error calculation.
- Developed the anomaly detection and threshold-based classification logic.
- Contributed to backend and dashboard integration.

- Performed functional testing, debugging, and verification.
- Evaluated system performance using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- Contributed to system deployment and GitHub project maintenance.
- Prepared and reviewed technical documentation, diagrams, results, and report sections.